

Vitta

Documento de Arquitetura do Sistema

Versão 1.0

Histórico de Revisão

Data	Versão	Descrição	Autor
20/04/2026	1.0	Criação/Inclusão de: - Descrição geral da arquitetura do sistema; - Estilo arquitetural adotado; - Principais componentes e suas responsabilidades; - Diagrama de arquitetura; - Tecnologias previstas; - Justificativa das decisões arquiteturais;	Enzo Gomide, Guilherme Ayala, Dimitri Vieira, Gustavo Anisio e Arthur Ribeiro

Sumário

1. Descrição Geral da Arquitetura do Sistema.....	3
2. Estilo Arquitetural Adotado.....	3
3. Principais Componentes e Responsabilidades.....	3
3.1. Camada de Visão (React):.....	3
3.2. Controladores de Requisição (NestJS):.....	3
3.3. Camada de Modelo e Persistência:.....	3
3.4. Componente de Geolocalização:.....	3
4. Diagrama de arquitetura.....	4
Diagrama 1 - Diagrama de Arquitetura do Sistema.....	4
5. Tecnologias Previstas.....	4
6. Visão Geral de Casos de Uso.....	5
Diagrama 2 - Diagrama do Principal Casos de Uso.....	5
7. Visão de Implantação.....	6
Diagrama 3 - Diagrama de Classes.....	6
Diagrama 4 - Diagrama de Sequência.....	6
8. Justificativa das Decisões Arquiteturais.....	7

1. Descrição Geral da Arquitetura do Sistema

O sistema **VITTA** utiliza o padrão arquitetural **MVC (Model-View-Controller)** para organizar a lógica da aplicação, garantindo a separação entre a interface do usuário, as regras de negócio e o acesso aos dados. Essa estrutura visa facilitar a manutenção e a escalabilidade do projeto, permitindo que alterações na interface não impactem diretamente a persistência dos dados e vice-versa.

2. Estilo Arquitetural Adotado

O estilo principal é o **MVC (Model-View-Controller)**:

- **Model:** Representado pelo **Prisma** e **PostgreSQL**, onde as entidades (como Doador, Agendamento, Instituição) e suas regras de persistência são definidas.
- **View:** Representada pelo **React**, que provê as interfaces interativas para os usuários. Em uma API REST, a "View" do servidor é o próprio JSON enviado ao cliente.
- **Controller:** Implementado através dos **Controllers do NestJS**, que recebem as requisições HTTP, validam os dados e acionam os serviços necessários.

3. Principais Componentes e Responsabilidades

3.1. Camada de Visão (React):

Gerencia o estado da interface (hooks como `useState` e `useMemo`) e renderiza os componentes de agendamento e histórico.

3.2. Controladores de Requisição (NestJS):

Gerenciam os endpoints da API (ex: `/api/doadores`), coordenando as ações entre a interface e o banco de dados.

3.3. Camada de Modelo e Persistência:

O **Prisma** atua como o mapeador de dados, garantindo que as operações no banco de dados respeitem os tipos definidos no **TypeScript**.

3.4. Componente de Geolocalização:

Responsável pela integração com APIs de mapas para fornecer a localização dos hemocentros próximos.

4. Diagrama de arquitetura

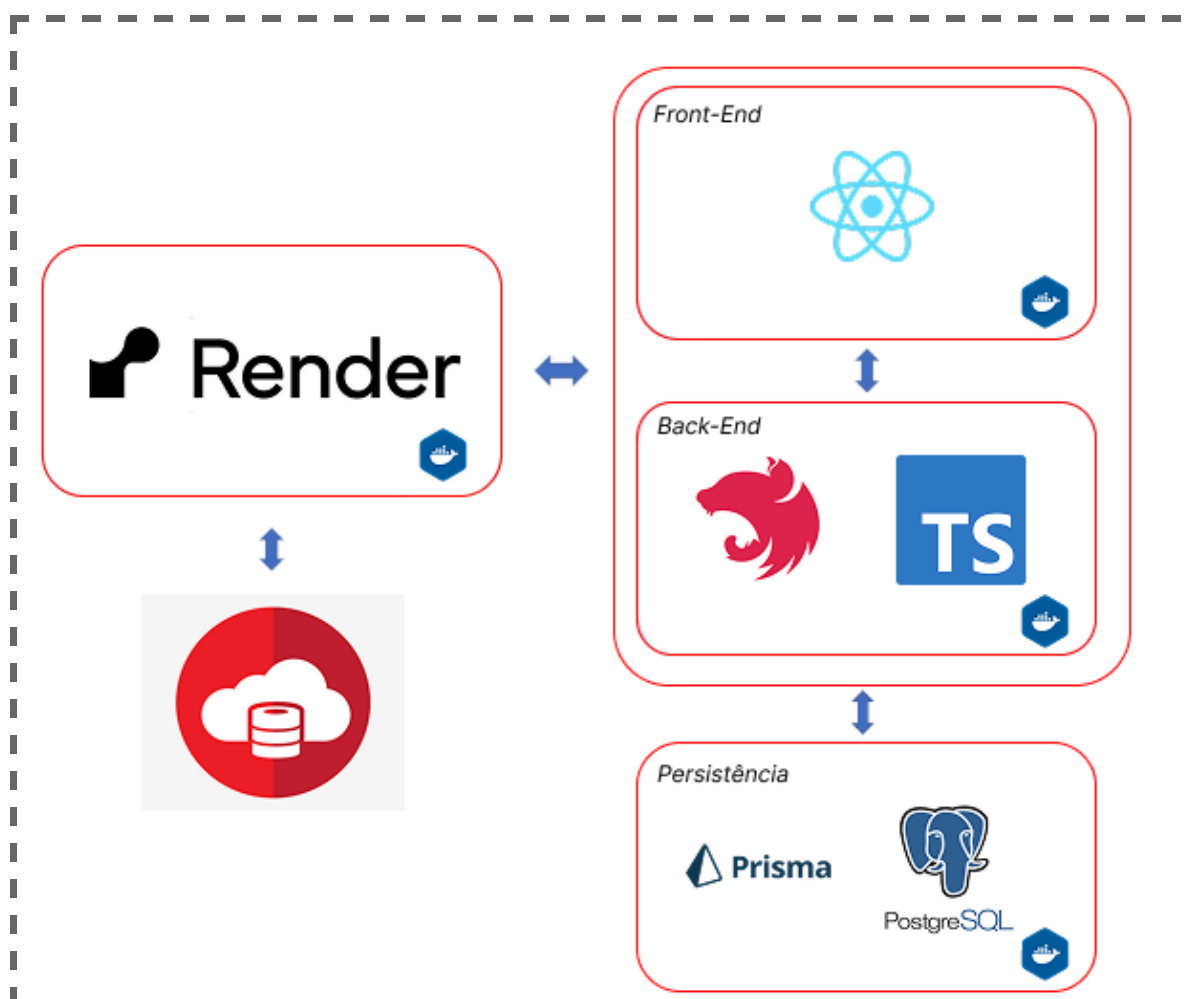


Diagrama 1 - Diagrama de Arquitetura do Sistema

5. Tecnologias Previstas

Tecnologia	Finalidade
React	Renderização da interface e experiência do usuário (UX).
NestJS (TS)	Orquestração da lógica de negócio e rotas da API.
PostgreSQL / Prisma	Definição do esquema de dados e persistência robusta.
Docker / Render	Padronização do ambiente e deploy automatizado.

6. Visão Geral de Casos de Uso

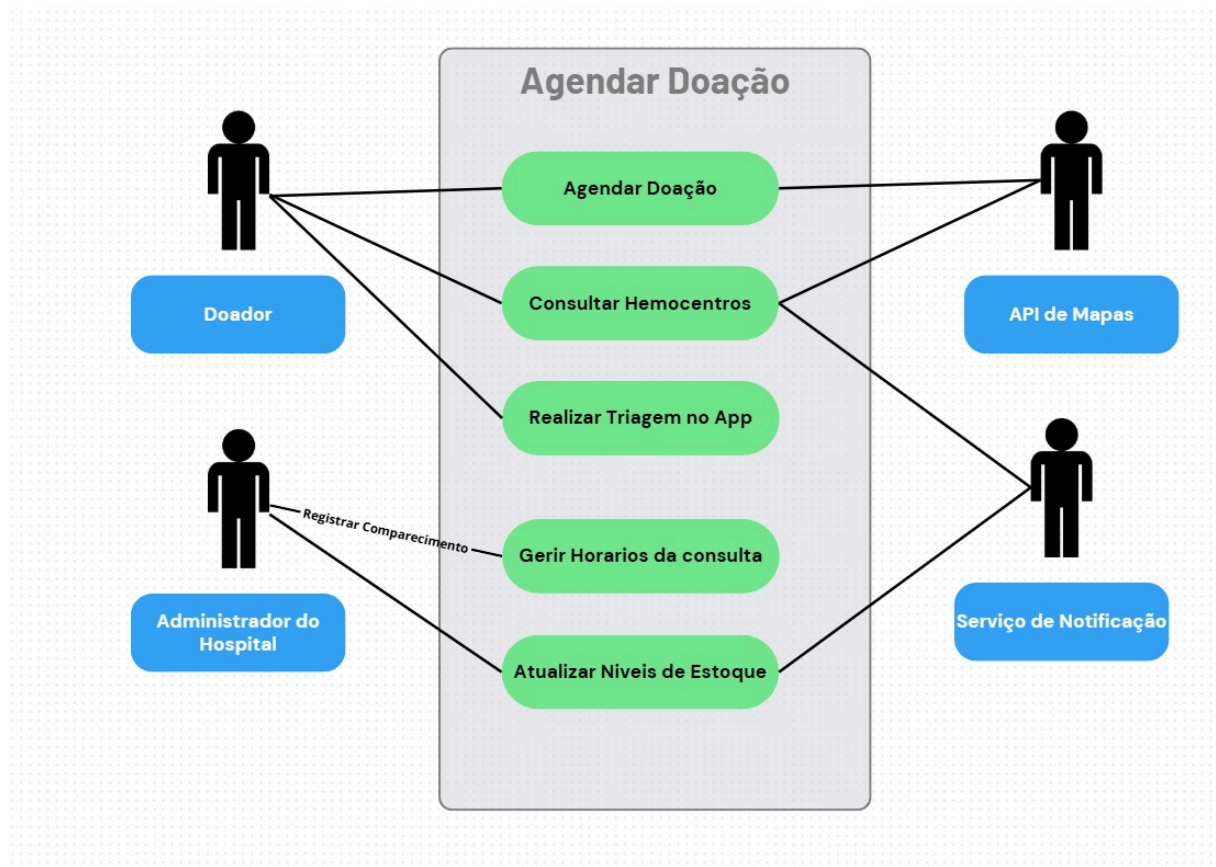


Diagrama 2 - Diagrama do Principal Casos de Uso

7. Visão de Implantação

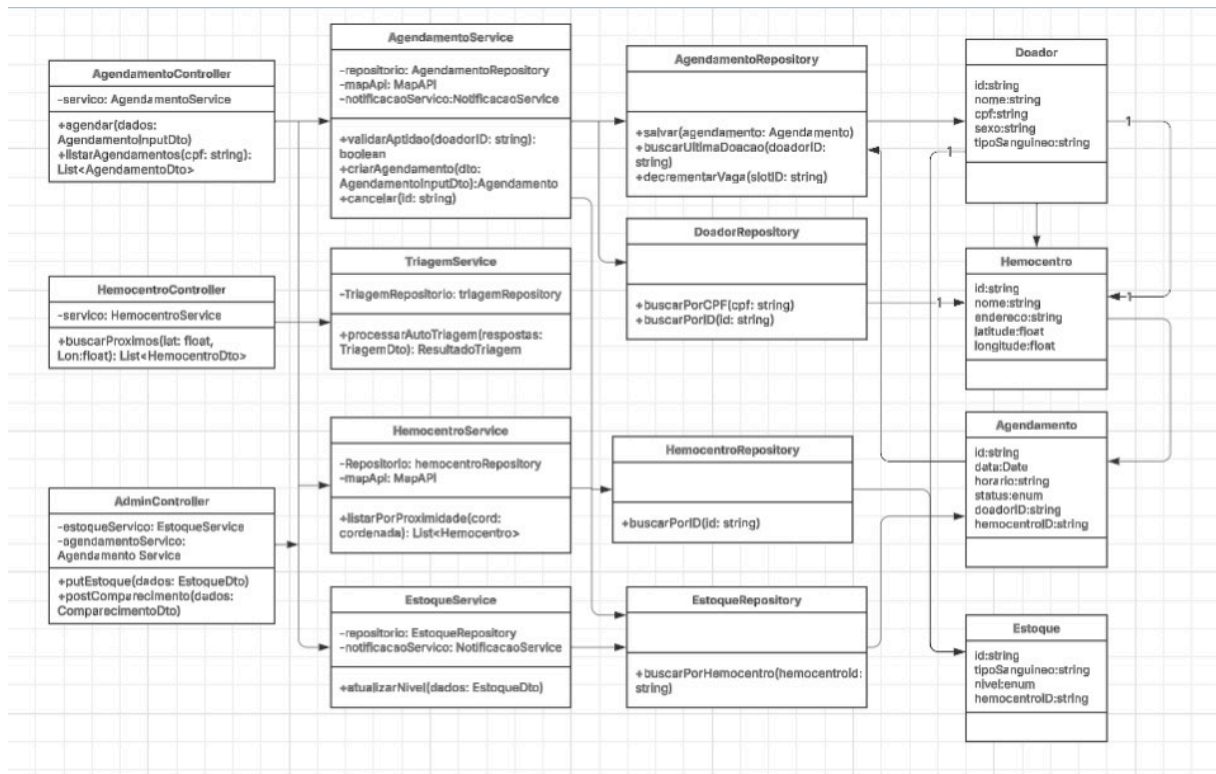


Diagrama 3 - Diagrama de Classes

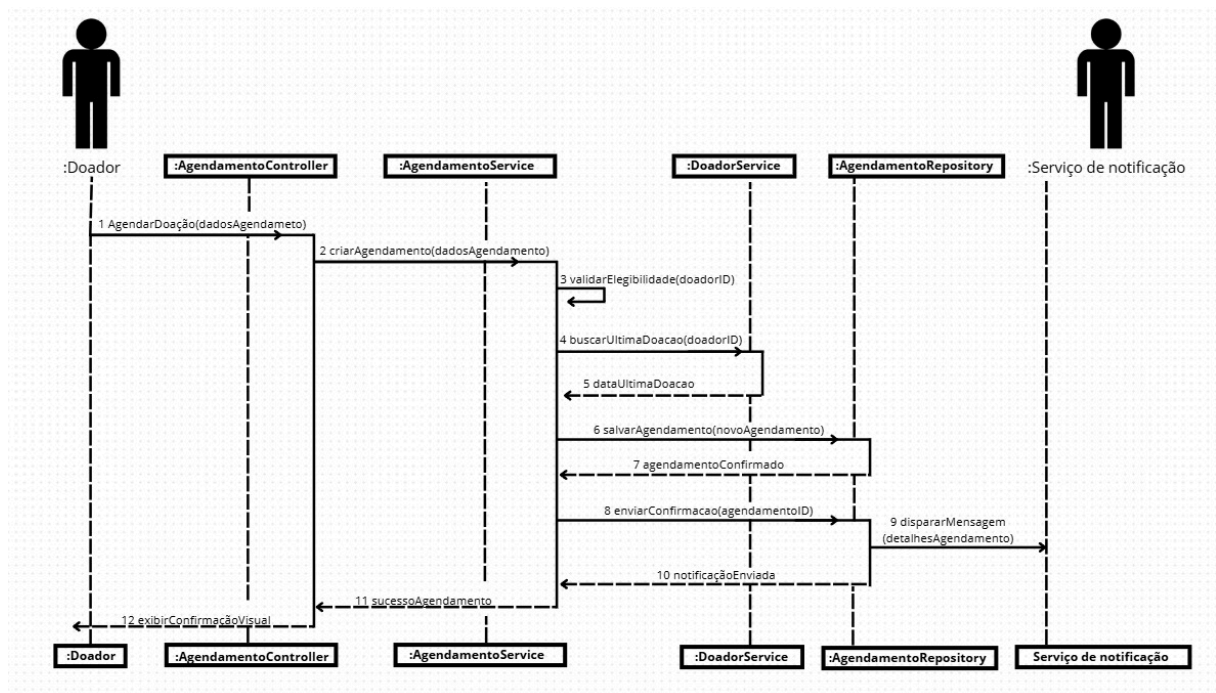
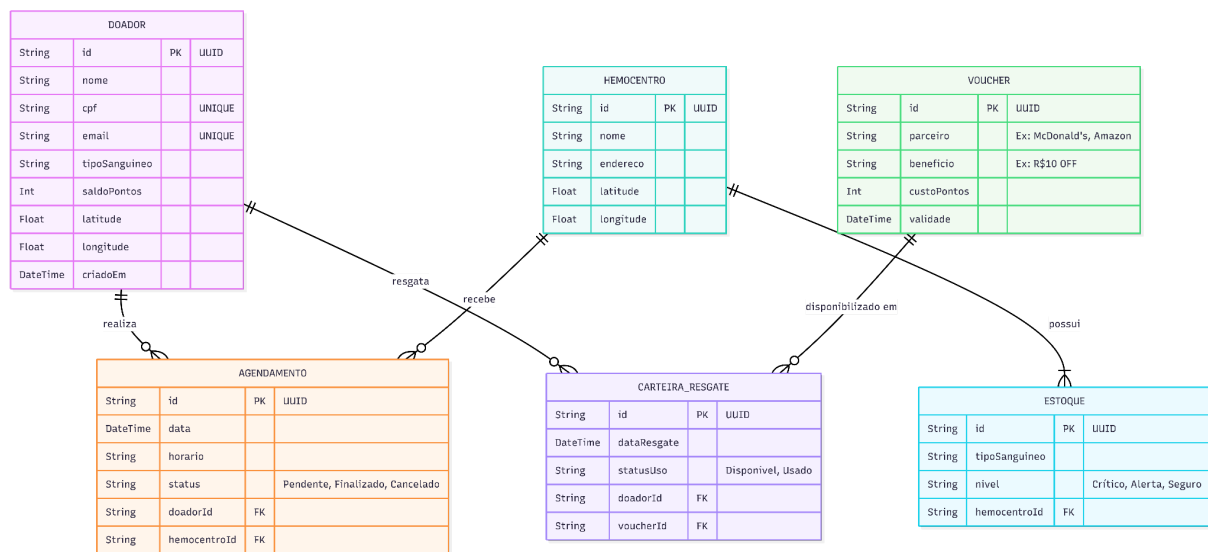


Diagrama 4 - Diagrama de Sequência

8. Modelo DER



A. Tabela DOADOR (Usuário)

É a entidade central do aplicativo.

- **Atributos Chave:** Além dos dados básicos (cpf, nome), ela armazena o saldoPontos (que será atualizado sempre que um agendamento for finalizado) e as coordenadas geográficas (latitude, longitude) para calcular a distância dos hospitais no mapa.

B. Tabela HEMOCENTRO

Representa os hospitais e postos de coleta.

- **Relacionamento com Agendamento (1:N):** Um hemocentro pode receber *vários* agendamentos, mas um agendamento pertence a apenas *um* hemocentro.
- **Relacionamento com Estoque (1:N):** Um hemocentro possui *vários* registros de estoque (um para cada tipo sanguíneo: A+, O-, etc.).

C. Tabela AGENDAMENTO

É a tabela de transação que liga o Doador ao Hemocentro.

- **Chaves Estrangeiras (FK):** Possui o doadorId e o hemocentroId.
- **Regra de Negócio:** O campo status é vital. Apenas quando o status mudar para "Finalizado" (ação feita pelo Administrador do hospital), uma Trigger ou o AgendamentoService irá adicionar pontos ao saldo do DOADOR.

D. Tabela ESTOQUE

Tabela de controle para o painel do Administrador.

- Ela é separada do hemocentro para facilitar consultas. Se o sistema precisar disparar uma notificação Push para doadores "O-", ele busca na tabela ESTOQUE onde tipoSanguineo == "O-" e nivel == "Crítico".

E. Tabelas VOUCHER e CARTEIRA_RESGATE

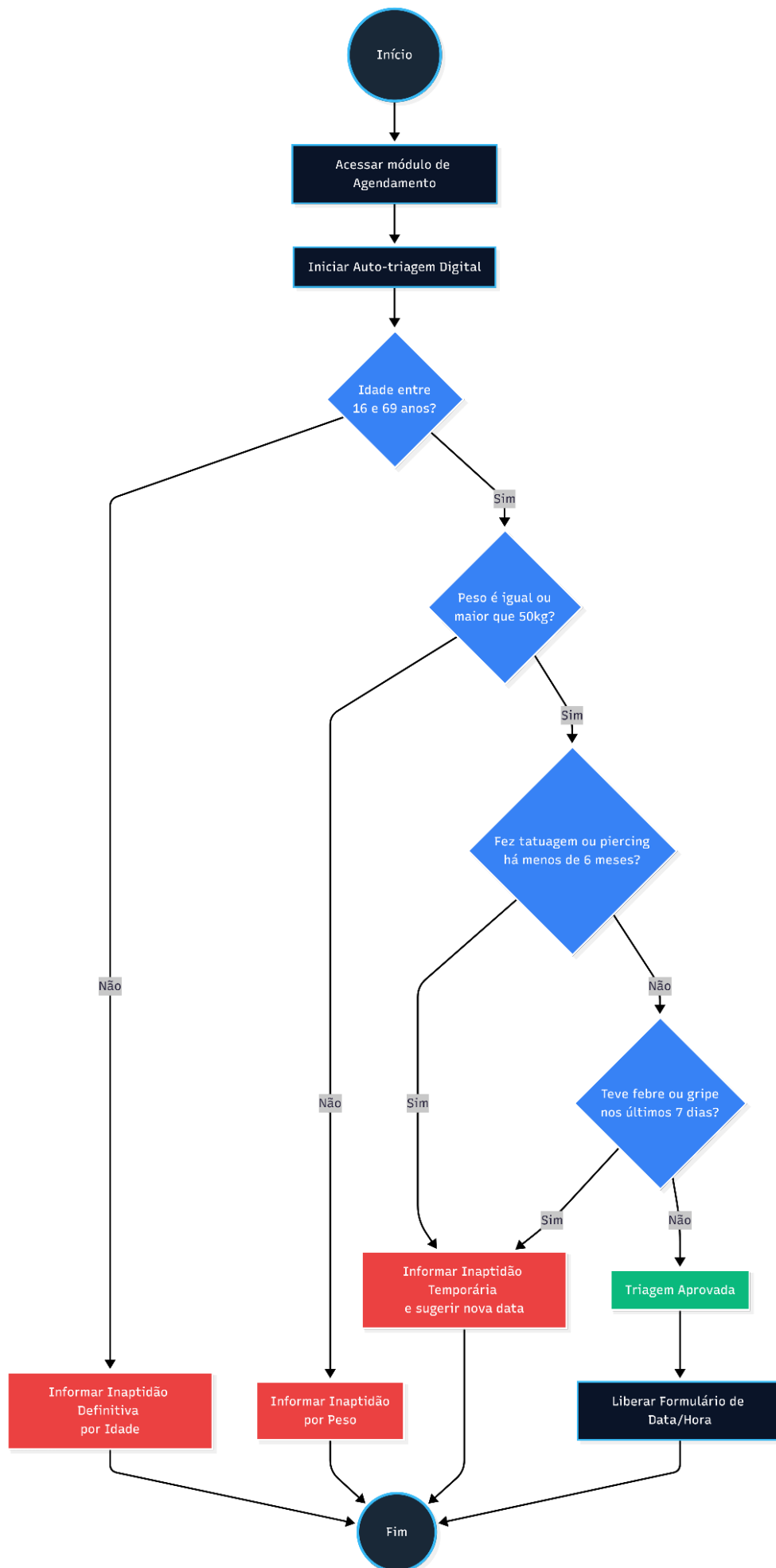
Representam o sistema de Gamificação (Aba "Minha Carteira").

- VOUCHER: É o catálogo de prêmios disponíveis (ex: Cupom Ifood, Desconto Amazon).
- CARTEIRA_RESGATE: É a tabela intermediária que resolve o relacionamento de Muitos-para-Muitos (N:M) entre Doador e Voucher. Quando um doador gasta seus pontos, um registro é criado aqui, contendo a dataResgate e o statusUso (para saber se ele já apresentou o cupom na loja ou não).

9. Diagrama de Atividade

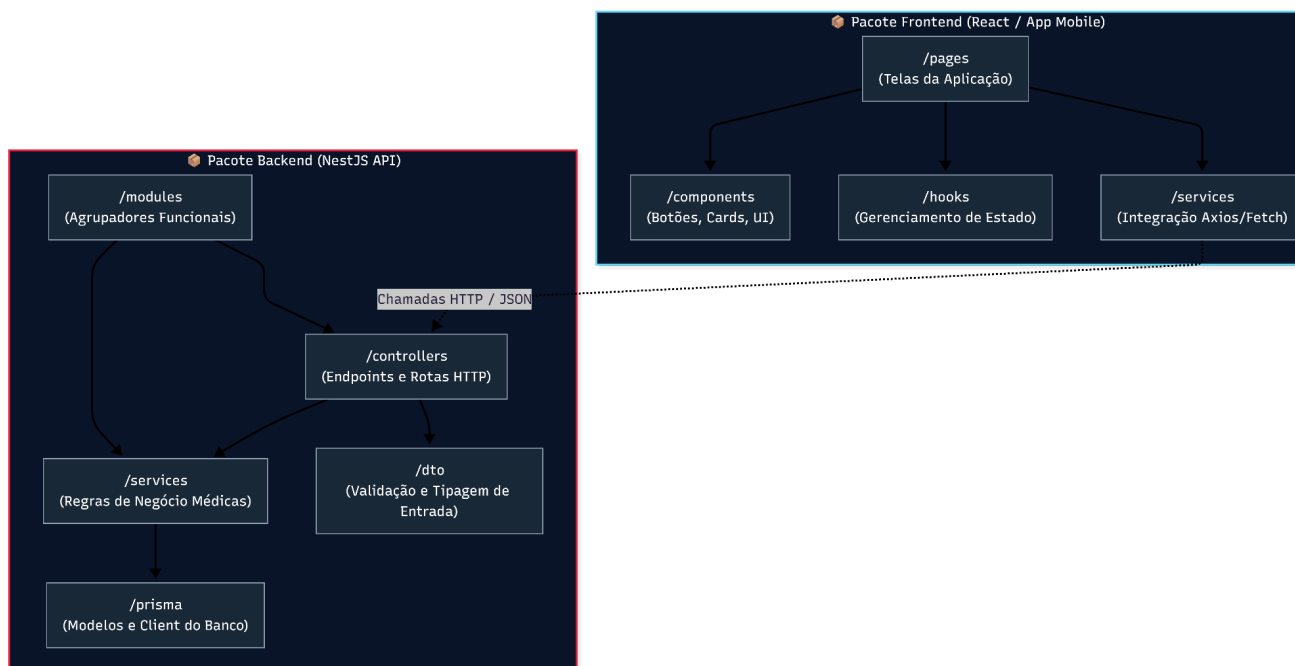
O Diagrama de Atividades acima traduz a complexidade das regras médicas de doação de sangue para um algoritmo simples de ser programado:

1. Nós de Ação (Retângulos Escuros): Representam os passos do usuário na interface, como acessar o módulo e iniciar o questionário.
2. Nós de Decisão (Losangos Azuis): Representam a inteligência do sistema. O sistema avalia:
 - Idade: Bloqueia menores de 16 ou maiores de 69.
 - Peso: Regra nacional básica (mínimo de 50kg para doar).
 - Tatuagem/Piercing: Analisa o risco de infecções recentes.
 - Sintomas (Febre/Gripe): Valida o bem-estar atual do doador.
3. Caminhos de Rejeição (Retângulos Vermelhos): Se qualquer resposta for negativa para os padrões de doação, o sistema desvia o usuário para uma tela de inaptidão. Notem que há uma diferença arquitetural importante aqui: inaptidão por idade ou peso é tratada como "Definitiva/Estrutural", enquanto tatuagem ou gripe gera uma inaptidão "Temporária", permitindo ao sistema sugerir uma data futura automaticamente.
4. Caminho de Aprovação (Retângulo Verde): Se o doador passar em todas as validações, a atividade principal de "Auto-triagem" é concluída com sucesso e o sistema libera o próximo passo: escolher o hemocentro e a data.



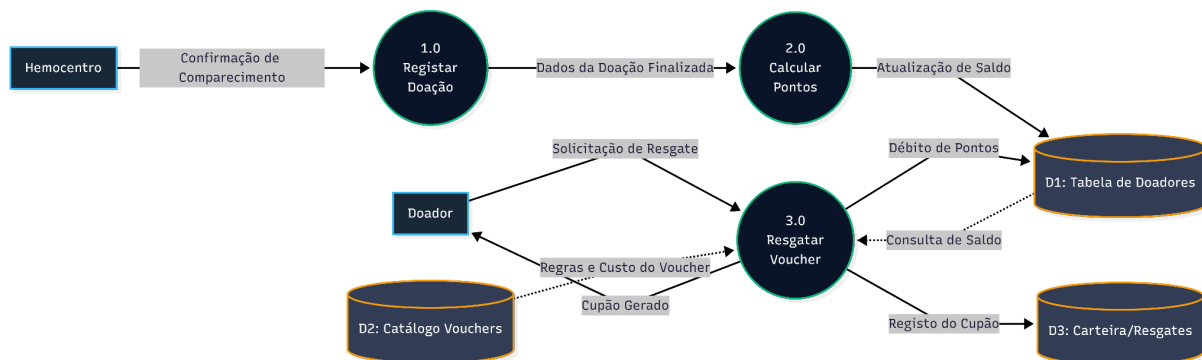
10. Diagrama de Pacotes

11. No Frontend: A organização isola as Views (/pages) dos blocos visuais genéricos (/components). O acesso à internet não é feito diretamente nas telas, mas delegado ao pacote de comunicação (/services), facilitando a manutenção caso a URL da API mude.
12. No Backend: Segue a hierarquia rigorosa do framework NestJS. O pacote /controllers atua como um porteiro: valida as requisições utilizando as regras do pacote /dto e repassa os dados limpos para os /services. O pacote /services é o único com permissão arquitetural para acionar o pacote /prisma, preservando o banco de dados contra acessos indevidos vindos diretamente das rotas web.



13. Diagrama de fluxo de dados

14. Processos 1.0 e 2.0 (Entrada e Processamento): Tudo começa quando a entidade externa (Hemocentro) confirma que a doação ocorreu. O processo 1.0 registra o ato, e o fluxo de dados passa para o processo 2.0, responsável por calcular a recompensa e guardar essa informação no depósito D1 (a tabela de Doadores na base de dados).
15. Processo 3.0 (Saída e Atualização): Quando o Doador quer um prêmio, interage com o processo 3.0. Este processo consome dados (linhas pontilhadas) dos depósitos D1 (para ver se há saldo suficiente) e D2 (para verificar quanto custa o prêmio). Após a validação lógica, o processo debita os pontos em D1, guarda a transação na carteira (D3) e, finalmente, entrega o fluxo de dados final (o Cupão/Voucher) à entidade Doador.



16. Justificativa das Decisões Arquiteturais

A escolha do padrão arquitetural MVC distribuído foi feita para garantir uma boa separação de responsabilidades no projeto. Com essa estrutura, as equipes de Front-End e Back-End conseguem trabalhar de forma mais independente, o que reduz bastante os conflitos na hora de integrar o código. Para fortalecer essa arquitetura, decidimos usar TypeScript em todas as camadas do sistema.

Na parte de banco de dados, escolhemos o Prisma ORM porque ele simplifica muito a criação e a atualização das tabelas (schemas). Ele funciona de forma rápida e segura entre o PostgreSQL e o React. O uso do Docker garante que todo o ambiente de desenvolvimento fique padronizado e isolado. Isso resolve de vez os problemas de incompatibilidade entre os computadores da equipe, garantindo que o sistema rode da mesma forma na máquina local e no ambiente de produção.